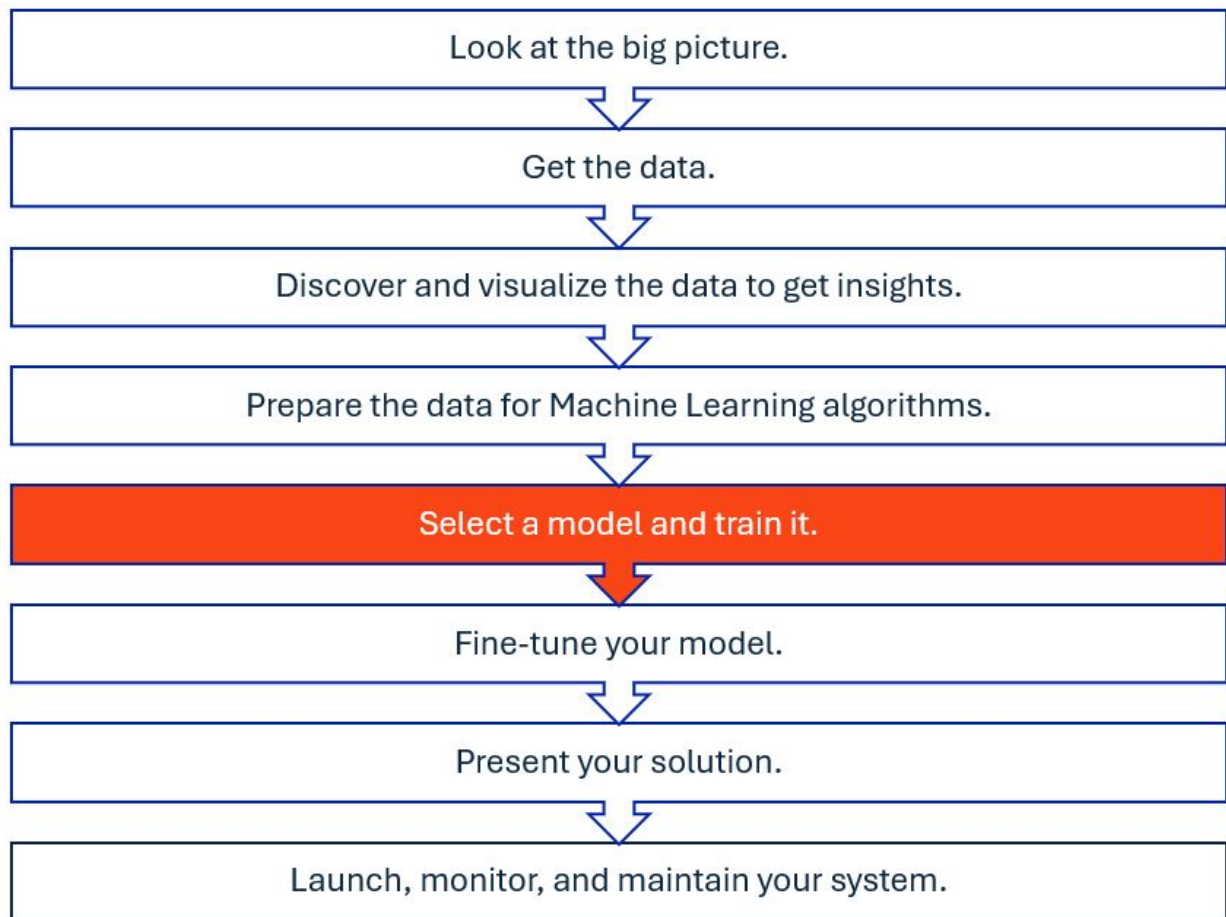

EEE5776. Applied Machine Learning



Module 2. Supervised Learning and Experimental Design

Overfitting vs. Underfitting
Ways to Avoid Overfitting
Cross-Validation
Regularization

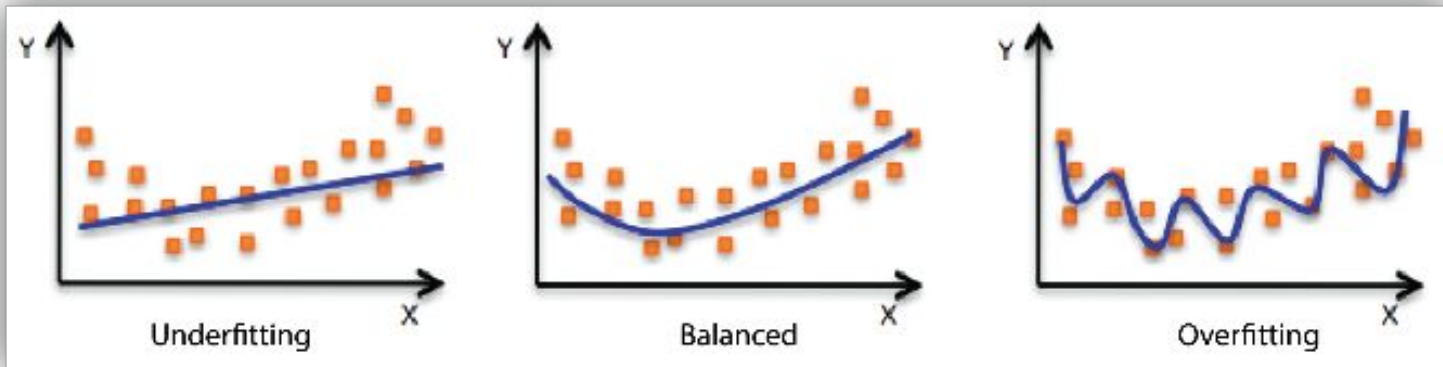
End-to-End Machine Learning Process



A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of parallel lines that curve and branch out.

Overfitting vs. Underfitting

Overfitting vs. Underfitting



A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of steps, set against a solid orange background.

Ways to Avoid Overfitting

Ways to avoid overfitting

- **Add more data!** - when possible
- **Occam's Razor** - select a model with lower complexity
- **Regularization** - constrains/regularizes the coefficients of the model
- **Cross-validation** - technique for utilizing the training data to fine-tune hyperparameter values

A series of white lines on the left side of the slide, consisting of several horizontal lines and a few diagonal lines that create a stepped, architectural effect.

Cross-Validation

Cross-Validation

The goal of **cross-validation**, or CV, is to test the model's ability to predict new data that was not used in estimating the model, in order to flag problems like overfitting or selection bias and to give an insight on how the model will generalize to an independent dataset (i.e., an unknown dataset).

Cross-validation allows us to determine the values for the hyperparameter that provide the most **generalization**. Cross-validation mitigates the occurrence of **overfitting**, but it does not eliminate it entirely.

Cross-validation strategies

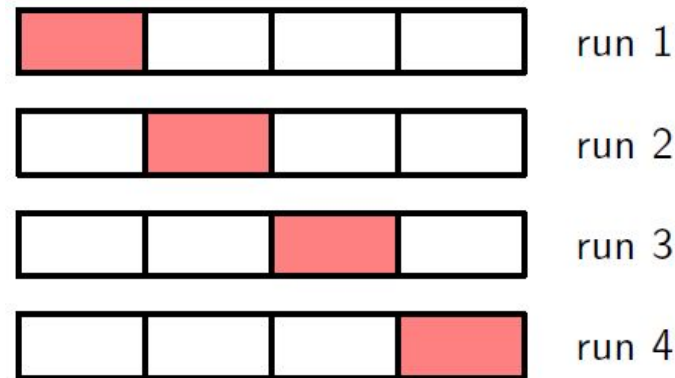
Let's consider the **MSE** as our performance measure. There are different strategies for implementing cross-validation:

1. **k-fold cross-validation.** Training set is partitioned into k folds.
2. **Leave-One-Out cross-validation.** Extreme case, where only one sample is left for validation.
3. **Stratified cross-validation.** Data is partition such that each class prior probability is preserved.
4. **$k \times 2$ cross-validation.** Training data is partitioned into 2 sets of equal size k times.

K-fold cross-validation

The technique of k -fold cross-validation, illustrated below for the case of $k=4$, involves taking the available data and partitioning it into k groups (in the simplest case these are of equal size). Then $k-1$ of the groups are used to train a set of models that are then evaluated on the remaining group. This procedure is then repeated for all k possible choices for the held-out group, indicated in the picture below by the red blocks, and the performance scores from the runs are then averaged.

K is typically 10 or 30. As K increases, the percentage of training instances increases and we get more robust estimators, but the validation set becomes smaller. Furthermore, there is the cost of training the classifier K times, which increases as K is increased. As N increases, K can be smaller; if N is small, K should be large to allow large enough training sets.



K-fold cross-validation

The following code randomly splits the training set into 10 distinct subsets called folds, then it trains and evaluates the Decision Tree model 10 times, picking a different fold for evaluation every time and training on the other 9 folds. The result is an array containing the 10 evaluation scores:

```
1 from sklearn.model_selection import cross_val_score
2
3 cross_val_score?
```

```
1 scores = cross_val_score(tree,
2                           X_train_prepared,
3                           t_train,
4                           cv=10,
5                           scoring='neg_mean_squared_error')
```

```
1 tree_rmse_scores = np.sqrt(-scores)
2
3 tree_rmse_scores
```

```
array([72519.47539568, 70859.42853898, 67850.27064217, 72674.23741557,
       70592.31591458, 75436.72072412, 71580.71964658, 72647.68498232,
       67759.42006596, 68810.52860776])
```

K-fold cross-validation

Scikit-Learn's cross-validation features expect a utility function (greater is better) rather than a cost function (lower is better), so the scoring function is actually the opposite of the MSE (i.e., a negative value), which is why the preceding code computes -scores before calculating the square root.

```
1 def display_scores(scores):
2     print('Scores:', scores)
3     print('Mean:', scores.mean())
4     print('Standard deviation:', scores.std())
5
6 display_scores(tree_rmse_scores)
```

```
Scores: [72519.47539568 70859.42853898 67850.27064217 72674.23741557
70592.31591458 75436.72072412 71580.71964658 72647.68498232
67759.42006596 68810.52860776]
Mean: 71073.08019337329
Standard deviation: 2306.446750110584
```

K-fold cross-validation

Let's compute the same scores for the Linear Regression model just to be sure:

```

1 lin_scores = cross_val_score(lin_reg,
2                               X_train_prepared,
3                               t_train,
4                               scoring='neg_mean_squared_error',
5                               cv=10)
6
7 lin_rmse_scores = np.sqrt(-lin_scores)
8
9 display_scores(lin_rmse_scores)

```

```

Scores: [71181.95098096 63825.56512744 67741.07241029 68256.29685279
66711.2271161 72240.73664717 69981.07976999 68438.51358706
65946.92067317 70073.85093767]
Mean: 68439.7214102642
Standard deviation: 2408.8928462974704

```

K-fold cross-validation

Let's compute the same scores for the Linear Regression model just to be sure:

```

1 lin_scores = cross_val_score(lin_reg,
2                               X_train_prepared,
3                               t_train,
4                               scoring='neg_mean_squared_error',
5                               cv=10)
6
7 lin_rmse_scores = np.sqrt(-lin_scores)
8
9 display_scores(lin_rmse_scores)

```

```

Scores: [71181.95098096 63825.56512744 67741.07241029 68256.29685279
66711.2271161 72240.73664717 69981.07976999 68438.51358706
65946.92067317 70073.85093767]
Mean: 68439.7214102642
Standard deviation: 2408.8928462974704

```

K-fold cross-validation

Let's try one last model now: the RandomForestRegressor. (We will study this model in detail later on in the course.)

Random Forests work by training many Decision Trees on random subsets of the features, then averaging out their predictions. Building a model on top of many other models is called Ensemble Learning, and it is often a great way to push ML algorithms even further.

```

1 from sklearn.ensemble import RandomForestRegressor
2
3 forest_reg = RandomForestRegressor()
4
5 forest_reg.fit(X_train_prepared, t_train)

```

RandomForestRegressor()

```

1 forest_scores = cross_val_score(forest_reg,
2                                 X_train_prepared,
3                                 t_train,
4                                 cv=5,
5                                 scoring='neg_mean_squared_error')
6
7 forest_rmse_scores = np.sqrt(-forest_scores)
8
9 display_scores(forest_rmse_scores)

```

Scores: [50067.09851428 49958.4537685 49554.81806239 51677.02608826
51431.09677114]
Mean: 50537.698640913375
Standard deviation: 850.7961664703095

A series of white lines on the left side of the slide, consisting of several horizontal lines that are slightly offset from each other, creating a stepped or comb-like effect.

Regularization

From our Linear Regression Model

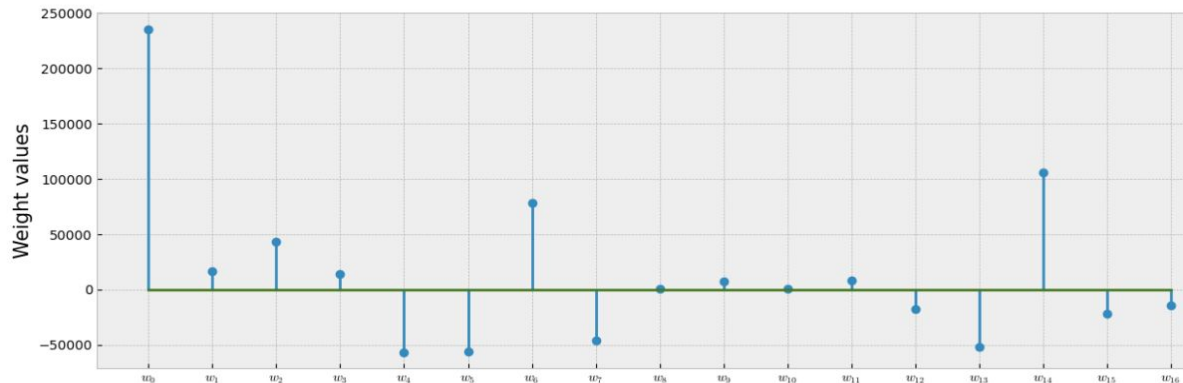
```
# concatenating all parameters into vector w
```

```
w = np.vstack((lin_reg.intercept_, lin_reg.coef_[0, np.newaxis]))
w
```

```
array([[235539.02199527],
       [ 16859.18615554],
       [ 42894.50706978],
       [ 13931.56938783],
       [-56657.82876645],
       [-56046.33896677],
       [ 78265.15994713],
       [-46114.16368891],
```



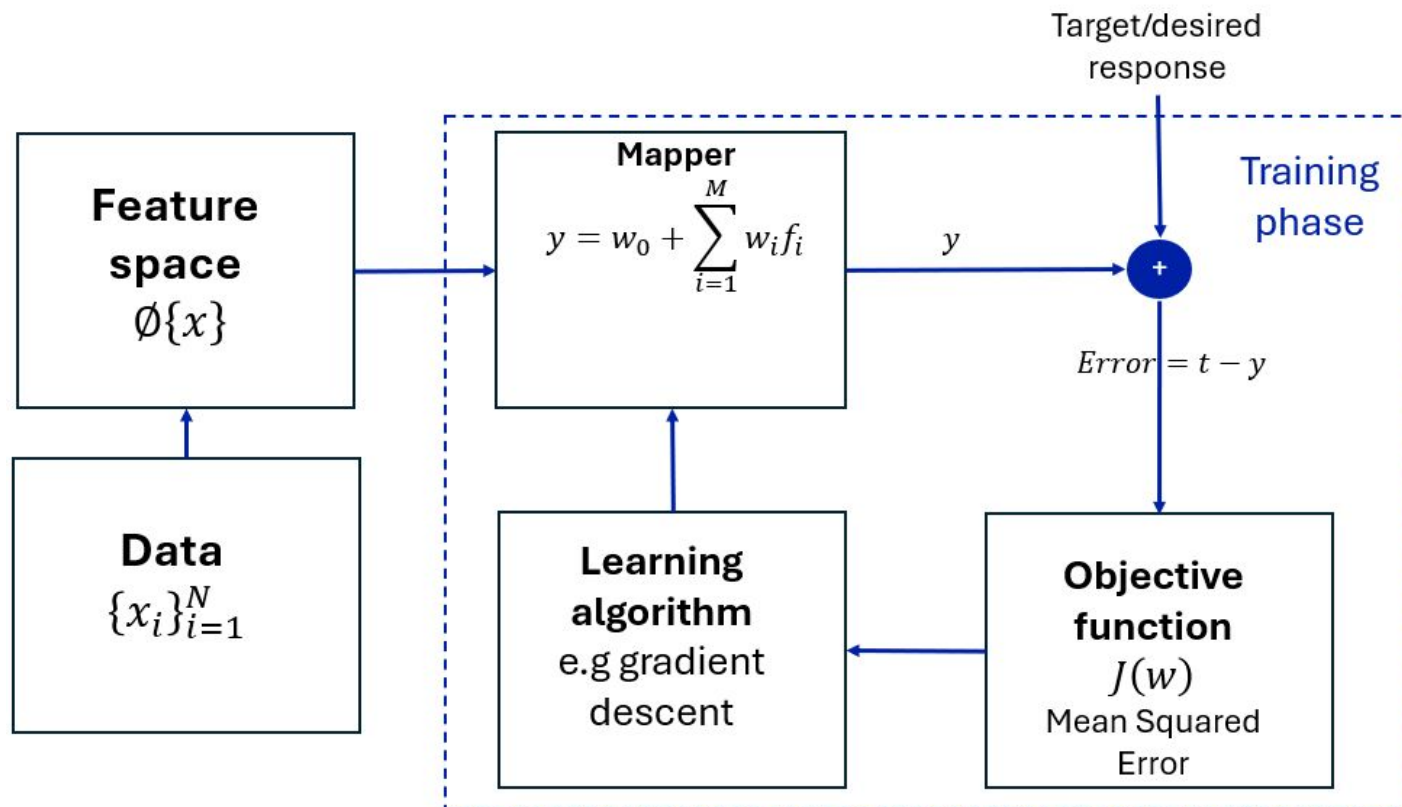
```
plt.figure(figsize=(15,5))
plt.stem(w)
plt.ylabel('Weight values', size=15)
plt.xticks(np.arange(len(w)), ['$w_{'+str(i)+'}$' for i in range(len(w))], rotation=0);
```



Types of regularizers

1. Ridge Regularizer
2. Lasso Regularizer
3. Elastic Net Regularizer

Supervised learning model



Let's go deeper - Linear regression

1. **Mapper:** a linear combination of features weighted by a set of parameters

$$y = f(\phi(x), \mathbf{w}) = \sum_{j=0}^M w_j \phi_j(x)$$

2. **Objective Function:** function that assesses the quality of the output of the model, for example, an error-based function such as L2-norm of the error or the mean-squared error (MSE) function:

$$J(w) = L(w) + \lambda R(w)$$

3. **Learning Algorithm:** an algorithm that solves or searches for the optimal values of the parameters $\mathbf{w}$ that optimize the objective function. It solves the following optimization problem:

$$\arg_{\mathbf{w}} \min J(\mathbf{w})$$

Ridge Regularizer

The Ridge regularizer, also known as the L2-Norm regularizer or weight decay, adds a term with the squared sum of the weights scaled by the hyperparameter λ .

This forces the learning algorithm to not only fit the data but also keep the model weights as small as possible. Note that the regularization term should only be added to the objective function during training. Once the model is trained, you want to evaluate the model's performance using the unregularized performance measure. The hyperparameter λ controls how much you want to regularize the model. If λ is 0 then the learning algorithm will just minimize the error term in the cost function. If λ is very large, then all weights end up very close to zero and the result is a flat line going through the data's mean.

$$\begin{aligned} R_{\mathbf{w}}^{(L2)} &= \lambda \sum_{i=0}^M w_i^2 \\ &= \lambda \|\mathbf{w}\|_2^2 \end{aligned}$$

Lasso Regularizer

The Lasso regularizer, also known as the L1-norm regularizer, it adds a term with the L1-norm of the weight vector $\mathbf{w}$ scaled by the hyperparameter λ . λ here has the same tradeoff responsibility.

An important characteristic of the Lasso regularizer is that it tends to eliminate the weights of the least important features by setting them to zero. This is known as sparsity.

$$\begin{aligned} R_{\mathbf{w}}^{(L1)} &= \lambda \sum_{i=0}^M |w_i| \\ &= \lambda \|\mathbf{w}\|_1 \end{aligned}$$

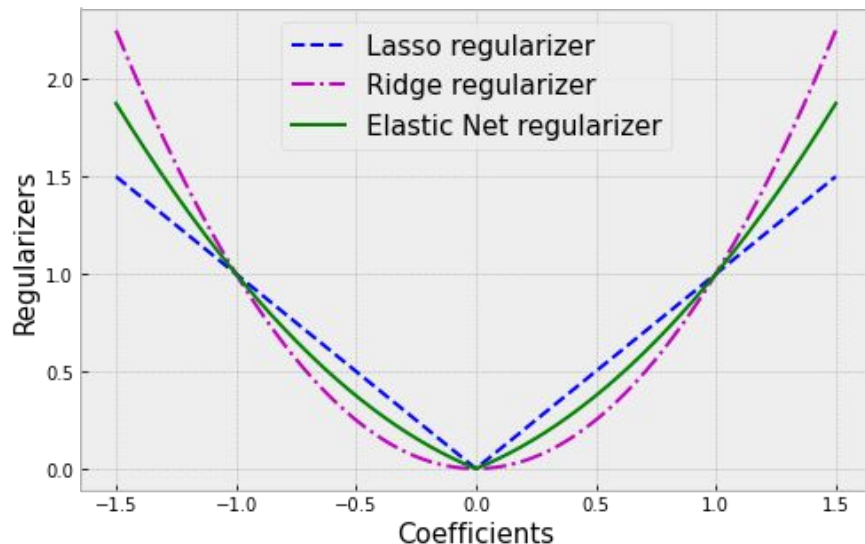
Elastic Net Regularizer

Elastic Net is a middle ground between Ridge and Lasso. The regularization term is a simple mix of both Ridge and Lasso's regularization terms, and you can control the mix ratio with the hyperparameter β .

$$\begin{aligned} R_{\mathbf{w}}^{(L12)} &= \beta \lambda \sum_{i=0}^M |w_i| + (1 - \beta) \lambda \sum_{i=0}^M w_i^2 \\ &= \beta R_{\mathbf{w}}^{(L1)} + (1 - \beta) R_{\mathbf{w}}^{(L2)} \end{aligned}$$

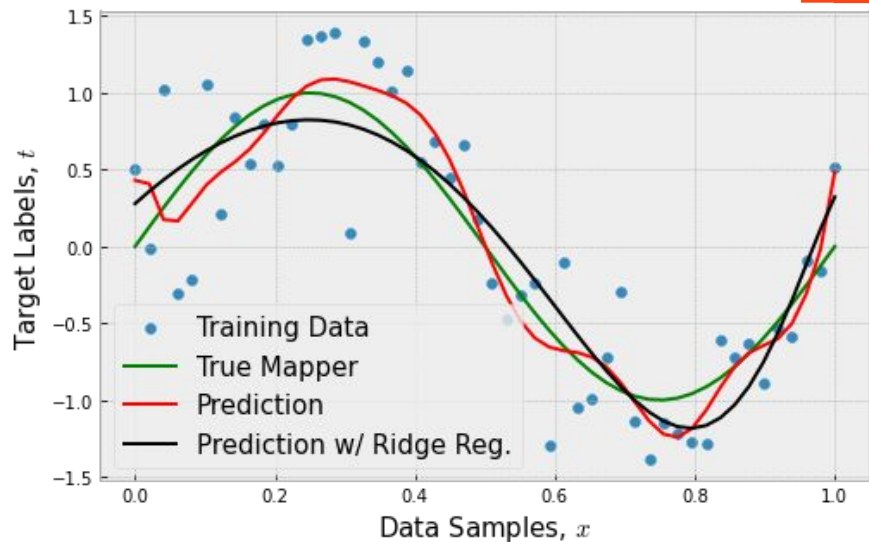
Big Picture

- Ridge regularizer: forces parameter values to be small but not equal to zero is highly affected by outliers
- Lasso regularizer: promotes sparsity, i.e. forces some parameter values to be exactly equal to zero it is not as affected by outliers Elastic
- Net regularizer: requires fine-tuning of an extra hyperparameter



Which one to choose?

Ridge is a good default, but if you suspect that only a few features are actually useful, you should prefer Lasso or Elastic Net since they tend to reduce the useless features' weights down to zero. In general, Elastic Net is preferred over Lasso since Lasso may behave erratically when the number of features is greater than the number of training instances or when several features are strongly correlated.



Implementation

```
# Creating a pipeline
ridge_pipeline = Pipeline([
    ('scaler', StandardScaler()), # Standardizes the features
    ('ridge', Ridge(alpha=1.0)) # Ridge regression with default alpha=1.0
])
```